

Secure Authentication Lab - Solution Guide

This document provides a comprehensive implementation guide for building a secure authentication service based on the lab requirements.

Step 1: Password Hashing & Server-Side Input Validation

1.1 Password Hashing: Passwords must never be stored in plain text. Use a strong library like *bcrypt* to hash passwords before storing them in the database.

```
// Node.js Example using bcrypt
const bcrypt = require('bcrypt');
const saltRounds = 10;
const hashedPassword = await bcrypt.hash(plainPassword, saltRounds);
```

1.2 Input Validation: Validate all incoming fields (e.g., checking if email is valid and password meets minimum character requirements) on the server side before handling logic.

```
// Server-side validation check
if (!email.includes('@') || password.length < 8) {
  return res.status(400).json({ error: 'Invalid input data' });
}
```

Step 2: Secure Cookie Settings, Logout, and Session Expiry

2.1 Secure Cookies: To prevent session hijacking and XSS attacks, set explicit attributes when issuing cookies.

```
res.cookie('token', jwtToken, {
  httpOnly: true, // Prevents client-side JS access
  secure: true, // Ensures cookie is sent over HTTPS only
  sameSite: 'strict', // Controls cross-site request behavior
  maxAge: 3600000 // 1 hour session expiry
});
```

2.2 Logout Endpoint: Implement a clear route to explicitly invalidate and clear the session cookie.

```
app.post('/logout', (req, res) => {
  res.clearCookie('token');
  return res.json({ message: 'Logged out successfully' });
});
```

Step 3: Rate Limits & Generic Authentication Errors

3.1 Rate Limiting: Protect auth routes (like `/login`) from brute-force automated attacks using tools like *express-rate-limit*.

```
const rateLimit = require('express-rate-limit');
const loginLimiter = rateLimit({
  windowMs: 15 * 60 * 1000, // 15 minutes window
  max: 5, // Limit each IP to 5 requests per window
  message: 'Too many login attempts, please try again later.'
});
app.use('/login', loginLimiter);
```

3.2 Generic Errors: Never disclose whether the email or password was the specific reason for failure. Use a unified message.

```
// Generic message for security obfuscation
return res.status(401).json({ error: 'Invalid email or password' });
```

Submission Requirements Checklist

Requirement	Status/Details
Repository Privacy	Private or View-only GitHub link
Documentation	README.md explaining how to run and test
Security Note	Brief write-up regarding the implemented controls